AcciSure — Complete Deployment Documentation

From Zero to Live: Vercel (Frontend) + Azure App Service (Backend) + Azure SQL

WHO THIS DOCUMENT IS FOR

This doc is written for you and anyone on your team who is a complete beginner to Azure. Every single step is here — creating accounts, what to click, what to fill, common mistakes, and how to fix them. Read it from start to finish before doing anything.

ARCHITECTURE OVERVIEW

Your final production setup looks like this:

User's Browser

↓

Angular App (Vercel) — https://accisure-frontend.vercel.app

↓ API calls

.NET 8 Backend (Azure App Service) — https://accisure-api.azurewebsites.net

↓ SQL queries

Azure SQL Database — accisure.database.windows.net,1433

- Frontend → Vercel (free, auto-deploys from GitHub, handles SPA routing perfectly)
- Backend → Azure App Service (hosts your .NET 8 API)
- Database → Azure SQL (cloud SQL Server)

PART 1 — BEFORE YOU TOUCH AZURE OR VERCEL

Fix #1: Remove All Hardcoded localhost URLs

This was one of the biggest mistakes in the project. Every service file had localhost:7140 hardcoded, which obviously breaks in production.

Files that needed to be fixed:

- auth.service.ts
- policy.service.ts
- agent.service.ts
- admin.service.ts
- chat.service.ts
- notification.service.ts

- claim.service.ts
- agent-dashboard.page.ts
- claims-officer-dashboard.page.ts

What to do: Create an Angular environment file and use it everywhere.

Create src/environments/environment.ts (for local dev):

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:7140/api'
};
```

Create src/environments/environment.prod.ts (for production):

```
export const environment = {
  production: true,
  apiUrl: 'https://accisure-api.azurewebsites.net/api'
};
```

In your angular.json, make sure fileReplacements is enabled for production builds:

```
"configurations": {
  "production": {
    "fileReplacements": [
      {
        "replace": "src/environments/environment.ts",
        "with": "src/environments/environment.prod.ts"
      }
    ]
  }
}
```

Then in every service file, replace hardcoded URLs like this:

```
// WRONG — DO NOT DO THIS
private apiUrl = 'http://localhost:7140/api/policy';


// CORRECT
import { environment } from '../environments/environment';
```

```
private apiUrl = `${environment.apiUrl}/policy`;
```

Fix #2: Fix CORS in Your .NET Backend

CORS was not configured properly for production. If CORS is wrong, your Angular app will get blocked by the browser when calling your Azure API — even if both apps are working perfectly.

In API/Program.cs, the CORS setup should read allowed origins from config, not be hardcoded:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
        policy
            .WithOrigins(
                builder.Configuration.GetSection("AllowedOrigins").Get<string[]>()
                ?? new[] { "http://localhost:4200" }
            )
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials()
    );
});

// Later in the pipeline:
app.UseCors("AllowFrontend");
```

Then in appsettings.json, add:

```
"AllowedOrigins": [
  "http://localhost:4200",
  "https://accisure-frontend.vercel.app"
]
```

When you later set up Azure App Service environment variables, you'll override this with your actual Vercel URL.

Fix #3: Fix Port Binding for Azure

Azure App Service does NOT use port 7140 or 5000. Azure injects its own port through the PORT environment variable (usually 8080). Your app must listen on whatever Azure tells it to.

In Program.cs, make sure you have this:

```
var port = Environment.GetEnvironmentVariable("PORT") ?? "8080";

builder.WebHost.UseUrls($"http://0.0.0.0:{port}");
```

Or more correctly for Azure App Service on Windows (which you're using), just don't lock the port — Azure handles it via IIS in Windows hosting. If you're on Linux plan, the above line is required.

If you see the app keep restarting or timing out on Azure, this is likely the cause.

Fix #4: Fix policy-config.json Path Resolution

PolicyService.cs had a bug where it could only find policy-config.json during local development (using relative paths). In Azure, the base directory is different.

Fix in PolicyService.cs:

```
// Find policy-config.json in both dev and production

string configPath;

var developmentPath = Path.Combine(Directory.GetCurrentDirectory(), "policy-config.json");

var productionPath = Path.Combine(AppContext.BaseDirectory, "policy-config.json");


if (File.Exists(developmentPath))

    configPath = developmentPath;

else if (File.Exists(productionPath))

    configPath = productionPath;

else

    throw new FileNotFoundException("policy-config.json not found");
```

Also, in Infrastructure.csproj, make sure this file gets copied to build output:

```
<ItemGroup>

  <None Update="policy-config.json">

    <CopyToOutputDirectory>Always</CopyToOutputDirectory>

  </None>

</ItemGroup>
```

PART 2 — FRONTEND DEPLOYMENT ON VERCEL

Vercel is the easiest option for Angular and you already have it working. This section confirms what settings are correct.

Step 1: Connect GitHub to Vercel

1. Go to https://vercel.com and sign up / log in (use GitHub login)
2. Click **Add New Project**
3. Click **Import Git Repository**
4. Authorize Vercel to access your GitHub account
5. Find your repo and click **Import**

Step 2: Configure Project Settings

Vercel will auto-detect Angular. Confirm these settings:

Setting	Value
Framework Preset	Angular
Root Directory	frontend-insurance (NOT the repo root — your Angular app is in a subfolder)
Build Command	npm run build
Output Directory	dist/frontend-insurance/browser

Why browser subfolder? Angular 17+ outputs SSR-style builds. The actual static files go inside dist/<project-name>/browser/. If you point Vercel to dist/frontend-insurance (without /browser), it won't find index.html and will fail.

Check your angular.json to confirm the output path:

"outputPath": "dist/frontend-insurance"

If that's what it says, the final browser files will be at dist/frontend-insurance/browser.

Step 3: Add Environment Variables in Vercel

In the Vercel project → **Settings** → **Environment Variables**, add:

Key	Value
GOOGLE_VISION_API_KEY	your key

Your Angular environment file handles the API URL at build time, so you don't need to add apiUrl here.

Step 4: Add vercel.json for SPA Routing

Without this file, refreshing any page other than / will give a 404. Create vercel.json in your frontend-insurance/ folder:

```
{
  "rewrites": [
```

```
  { "source": "/(.*)", "destination": "/index.html" }
 ]
}
```

Step 5: Deploy

Click Deploy. Vercel will build and deploy. Your URL will be something like: https://accisure-frontend.vercel.app

Every time you push to the master branch on GitHub, Vercel redeploys automatically.

PART 3 — AZURE ACCOUNT SETUP (COMPLETE BEGINNER START)

Step 1: Create Azure Account

1. Go to https://azure.microsoft.com
2. Click **Start Free**
3. Sign in with your Microsoft account (or create one)
4. Fill in your details — you need a credit/debit card for verification but the Free tier won't charge you
5. Azure gives you $200 free credit for 30 days + always-free services

Step 2: Access Azure Portal

After account creation, go to: https://portal.azure.com

You'll see the Azure Portal dashboard. This is where you manage everything.

PART 4 — CREATE AZURE SQL DATABASE

Step 1: Create a Resource Group First

A Resource Group is just a folder that holds all your Azure resources together.

1. In the portal search bar, type **Resource groups**
2. Click **Create**
3. Fill:
    - Subscription: Azure subscription 1
    - Resource group name: accisure-rg
    - Region: Central India (or closest to you)
4. Click **Review + Create** → **Create**

Step 2: Create the SQL Server and Database

1. Click **Create a resource** (top left)

2. Search for **Azure SQL**
3. Select **Azure SQL** → **Create**
4. Choose **SQL databases** → **Single database** → **Create**

Fill in the form:

Field	Value
Subscription	Azure subscription 1
Resource group	accisure-rg
Database name	AcciSureDB
Server	Click "Create new"

For the new server:

- Server name: accisure-server (this becomes accisure-server.database.windows.net)
- Location: Central India
- Authentication method: SQL authentication
- Server admin login: adminuser
- Password: Choose something strong like AcciSure@2025!

Back on the main form:

- Compute + storage: Basic (for testing) — cheapest option
- Backup storage: Locally redundant

5. Click **Review + Create** → **Create**

Wait 2-3 minutes for the database to be created.

Step 3: Configure Firewall Rules

By default, Azure SQL blocks all connections. You need to allow access.

1. Go to your SQL Server resource (not the database — the server): accisure-server
2. Click **Networking** in the left menu
3. Under Firewall rules:
    - Toggle **Allow Azure services and resources to access this server** → ON
    - Click **Add your client IP** (to allow access from your machine for running migrations)
4. Click **Save**

Step 4: Get the Connection String

1. Go to your database: AcciSureDB

2. Click **Connection strings** in the left menu
3. Copy the **ADO.NET** connection string — it looks like:

Server=tcp:accisure-server.database.windows.net,1433;Initial Catalog=AcciSureDB;Persist Security Info=False;User ID=adminuser;Password={your_password};MultipleActiveResultSets=False;Encrypt=True;TrustServerCertificate=False;Connection Timeout=30;

Replace {your_password} with your actual password. Save this string — you'll need it in the next section.

PART 5 — CREATE AZURE APP SERVICE (BACKEND)

Step 1: Create the Web App

1. In Azure Portal, click **Create a resource**
2. Search **Web App** and select it
3. Click **Create**

Fill in the form:

Field	Value
Subscription	Azure subscription 1
Resource group	accisure-rg
Name	accisure-api
Publish	Code
Runtime stack	.NET 8 (LTS)
Operating System	Windows
Region	Central India
Pricing plan	F1 Free (for testing) or B1 Basic (for production)

4. Click **Review + Create** → **Create**

After creation, your backend URL will be: https://accisure-api.azurewebsites.net

Step 2: Configure Environment Variables

This is the most critical step. All your secrets and config values go here — never hardcode them in the code.

1. Go to your App Service: accisure-api
2. In the left menu: **Settings** → **Environment variables**

3. Click **+ Add** for each of the following:

Connection String (add under "Connection strings" tab, not "App settings"):

Name	Value	Type
DefaultConnection	(your full Azure SQL connection string from Part 4)	SQLAzure

App Settings (add under "App settings" tab):

Name	Value
AllowedOrigins__0	https://accisure-frontend.vercel.app
JWT__SecretKey	(generate a long random string, min 32 chars)
JWT__Issuer	InsuranceAPI
JWT__Audience	InsuranceClient
ImageKit__PublicKey	(from your ImageKit dashboard)
ImageKit__PrivateKey	(from your ImageKit dashboard)
ImageKit__UrlEndpoint	(from your ImageKit dashboard)
ExternalApis__Groq__ApiKey	(your Groq API key)
ExternalApis__ElevenLabs__ApiKey	(your ElevenLabs key)
ExternalApis__Deepgram__ApiKey	(your Deepgram key)
ASPNETCORE_ENVIRONMENT	Production

4. Click **Save** (important — it won't apply until saved)
5. Click **Restart** to restart the app service

How Azure maps these to your appsettings.json:

In .NET, AllowedOrigins__0 maps to AllowedOrigins[0] in JSON. The double underscore __ is Azure's way of representing nested config and arrays. So your C# code that reads Configuration["AllowedOrigins"] as a string array will automatically pick this up.

PART 6 — DEPLOY BACKEND CODE TO AZURE

Method 1: Visual Studio (Easiest — Recommended)

1. Open your InsurancePlatform solution in Visual Studio 2022
2. In Solution Explorer, right-click on the **API project** (not the solution — the actual API project)
3. Click **Publish**
4. Select **Azure** → **Next**

5. Select **Azure App Service (Windows)** → **Next**
6. Sign in with your Azure account if prompted
7. Select your subscription → expand → find accisure-api under accisure-rg
8. Click **Finish** → **Publish**

Visual Studio will automatically build, package, and upload the app. You'll see a green checkmark when it's done.

Method 2: GitHub Actions (CI/CD — For Ongoing Deployments)

Once the above Visual Studio publish has been done once, set up GitHub Actions for automatic deployment.

1. In Azure Portal → your App Service → **Deployment Center**
2. Source: **GitHub**
3. Authorize GitHub → select your repo → select master branch
4. Azure will automatically create a .github/workflows/ file in your repo

From now on, every push to master will automatically redeploy your backend.

PART 7 — RUN DATABASE MIGRATIONS

Before your app can work, the database tables need to be created.

Option A: From Your Local Machine

Make sure your appsettings.json (local) has the Azure SQL connection string temporarily:

```
cd your-solution-folder

dotnet ef database update --project Infrastructure --startup-project API
```

Then revert appsettings.json back so you don't accidentally commit the production connection string.

Option B: Auto-migrate on Startup (Safer)

In Program.cs, add this before app.Run():

```
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.Migrate(); // runs pending migrations automatically
}
```

This way, every time the app starts, it checks for pending migrations and applies them.

PART 8 — VERIFY EVERYTHING IS WORKING

Test 1: Backend Health Check

Open your browser and go to: https://accisure-api.azurewebsites.net/swagger

If Swagger UI opens → your backend is running correctly.

If you get a 500 error or blank page → check the App Service logs:

1. Azure Portal → App Service → **Log stream** (under Monitoring)
2. Read the error and fix it

Test 2: Database Connection

In Swagger, try hitting any GET endpoint that reads from the database. If it returns data, the database connection is working.

If it fails with "Cannot open server" → go back to Step 3 of Part 4 and make sure **Allow Azure services** is turned on in SQL firewall.

Test 3: Frontend Talks to Backend

1. Open your Vercel URL: https://accisure-frontend.vercel.app
2. Try logging in
3. Open browser DevTools → Network tab
4. Check that API calls are going to https://accisure-api.azurewebsites.net/api/... (NOT localhost)
5. Check that no 401/403/CORS errors appear in the console

If you see a CORS error like:

Access to XMLHttpRequest at 'https://accisure-api...' from origin 'https://accisure-frontend...' blocked

→ Go back to Part 1 Fix #2 and confirm AllowedOrigins__0 in Azure environment variables matches your Vercel URL exactly (with https, no trailing slash).

PART 9 — COMMON MISTAKES AND HOW TO FIX THEM

Mistake 1: Wrong Output Directory in Vercel

Problem: Vercel deploys but you see a blank page or 404.

Cause: Angular 17+ puts final files in dist/<name>/browser/ not dist/<name>/. Vercel was pointed at the wrong folder.

Fix: In Vercel project settings → Output Directory → set to dist/frontend-insurance/browser

Also confirm in angular.json:

"outputPath": "dist/frontend-insurance"

Mistake 2: 404 on Page Refresh (SPA Routing Broken)

Problem: Navigating to /dashboard works from the app, but if you refresh the page you get 404.

Cause: Vercel (and any static host) doesn't know about Angular's client-side routes. It tries to find a file called dashboard on the server and fails.

Fix: Add vercel.json (shown in Part 2 Step 4) or netlify.toml depending on your host. This tells the server to always serve index.html and let Angular handle routing.

Mistake 3: CORS Errors in Production

Problem: App works locally but in production the browser console shows CORS blocked errors.

Cause 1: AllowedOrigins in Azure env vars doesn't match the Vercel URL exactly.

Fix: Check for typos. The URL must be exactly https://accisure-frontend.vercel.app — no trailing slash, exact case.

Cause 2: app.UseCors() is placed in the wrong order in Program.cs.

Fix: CORS middleware must come BEFORE UseAuthentication, UseAuthorization, and MapControllers:

```
app.UseCors("AllowFrontend");     // Must be first

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();
```

Mistake 4: App Service Keeps Restarting

Problem: Azure App Service shows "Application Error" or keeps cycling restarts.

Cause 1: Port binding issue (Linux plan only). Your app binds to a fixed port instead of reading from environment.

Fix: Add to Program.cs:

```
var port = Environment.GetEnvironmentVariable("PORT") ?? "8080";

builder.WebHost.UseUrls($"http://0.0.0.0:{port}");
```

Cause 2: Missing environment variables. If code tries to read a config key that doesn't exist and throws an exception at startup, the app crashes.

Fix: Check the Log Stream in Azure Portal for startup errors.

Cause 3: Startup time exceeded. Free tier (F1) has a very short startup timeout.

Fix: Upgrade to B1 plan, or make sure no heavy operations run synchronously at startup.

Mistake 5: Database Migration Fails

Problem: dotnet ef database update throws "Cannot open server" or "Login failed".

Fix checklist:

- Your IP must be in the SQL Server firewall rules
- The password in the connection string must not have special characters that break the connection string format (if your password contains ; or =, you must wrap it in quotes in the connection string)
- Make sure TrustServerCertificate=False and Encrypt=True are in the connection string (Azure SQL requires encryption)

Mistake 6: Swagger Not Loading in Production

Problem: Swagger UI shows in local dev but 404 in Azure.

Cause: Swagger is conditionally enabled only in Development environment.

Fix: In Program.cs, either:

```
// Show Swagger in all environments (for testing)
app.UseSwagger();
app.UseSwaggerUI();
```

Or explicitly allow it in production:

```
if (app.Environment.IsDevelopment() || app.Environment.IsProduction())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

Mistake 7: SignalR WebSocket Fails on Azure

Problem: SignalR works locally but disconnects or fails on Azure App Service.

Cause: Azure App Service has WebSocket support disabled by default.

Fix:

1. Azure Portal → App Service → **Configuration** → **General settings**
2. Scroll down to find **Web sockets** → Toggle to **On**
3. Click **Save** → **Restart**

Mistake 8: policy-config.json Not Found

Problem: App throws FileNotFoundException: policy-config.json not found in production.

Cause: The file path works in local dev but not after deployment because the working directory changes.

Fix: Already covered in Part 1 Fix #4 — use AppContext.BaseDirectory as fallback and make sure the file has CopyToOutputDirectory: Always in the .csproj.

PART 10 — CHECKLIST BEFORE GOING LIVE

Use this before every deployment:

Code checks:

- No localhost references anywhere in Angular services
- environment.prod.ts has correct Azure API URL
- No API keys hardcoded in frontend code
- vercel.json exists with SPA redirect rule

Azure checks:

- All environment variables set in App Service
- SQL firewall allows Azure services
- Web Sockets enabled (for SignalR)
- CORS origin matches Vercel URL exactly
- App Service has been restarted after env var changes

Database checks:

- Migrations have been applied
- Connection string in Azure env vars is correct (password replaced, not {your_password})

Test checks:

- Swagger opens at /swagger
- Login works end-to-end
- No CORS errors in browser console
- Page refresh doesn't give 404 on any route

QUICK REFERENCE — YOUR PROJECT URLS

Service	URL
Frontend (Vercel)	https://accisure-frontend.vercel.app
Backend (Azure)	https://accisure-api.azurewebsites.net
Backend Swagger	https://accisure-api.azurewebsites.net/swagger
Azure SQL Server	accisure-server.database.windows.net,1433
Azure Portal	https://portal.azure.com
Vercel Dashboard	https://vercel.com/dashboard

That's the full end-to-end guide based on everything from your chat history. The biggest issues were: wrong Netlify/Vercel publish directory (missing /browser), hardcoded localhost URLs across all service files, CORS not reading from config properly, and policy-config.json path resolution breaking in production. All of those are documented above with the exact fixes so next time you or anyone else can start fresh and not hit the same walls.